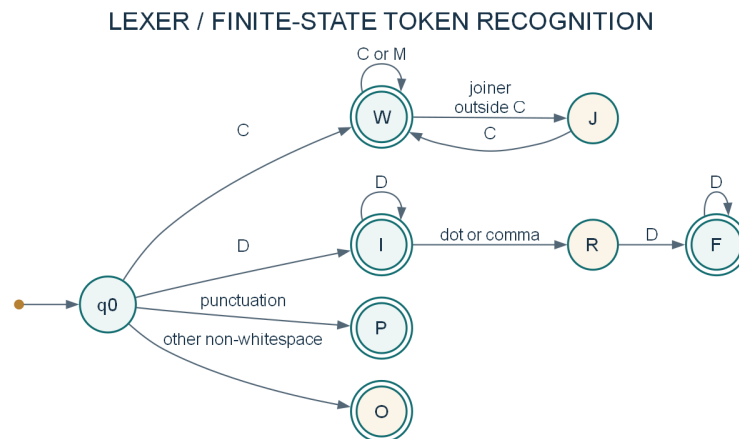


Our two automata, made simple

An **automaton** is a machine that follows rules to read input.

1 The lexer: finding the pieces

The lexer cuts text into **tokens**: words, numbers and punctuation. Its **DFA** (deterministic finite automaton) tracks the current state; it has no stack.



Reading the arrows: C = a letter-like word character; D = a digit; M = a combining accent mark. A double circle means a complete token can end here.

How it works

1. **Start at q0.** A letter leads to W (word), a digit to I (integer), punctuation to P, and another non-space character to O.
2. **Follow the arrows.** Stay in W for more letters, or I/F for more digits. J waits for a letter after an apostrophe or hyphen. R waits for a digit after a decimal point or comma.
3. **Keep the longest complete token and restart.** Skip spaces. J and R are waiting states, so a token cannot finish there.

Three small examples

Input	What happens
j'ai	j: W; apostrophe: J; a and i: W. Output: one word.
3.14	3: I; dot: R; 1 and 4: F. Output: one number.
3.	No digit follows the dot. Output: number 3, then punctuation .

What goes to the parser?

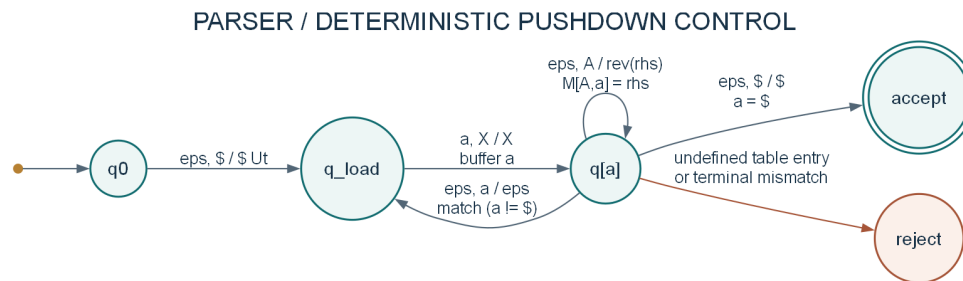
After splitting `je suis kass`, vocabulary classification labels the three tokens **French function word**, **verb**, **adjective**. The parser receives these categories.

Python regular expressions perform the recognition shown above. Classification is a separate step: a word-shaped token can still be UNKNOWN.

Main files: `compiler/lexer/tokenizer.py` and `compiler/lexer/lexicon.py`.

2 The parser: checking the order

The **pushdown automaton** adds a **stack**: a last-in, first-out to-do list. Our **LL(1) parser** reads categories from left to right, using one category of lookahead to choose a grammar rule.



States: q0 starts; q_load gets the next category; q[a] holds category a while the parser works. \$ marks the end; **eps** means no input is read on that arrow.

The three main actions

1. **Expand.** Start with [$\$, Ut$] (Ut = whole utterance). Replace a grammar symbol with its table rule, pushed backwards so the first symbol is on top. Do not read a token yet.
2. **Match.** Compare the expected category on top with the current input category. If equal, remove the top and move to the next token.
3. **Finish.** Empty rules just remove a grammar symbol. Accept when only \$ remains and input has ended. A mismatch or missing rule means reject.

Diagram notation: $M[A,a]$ selects a rule for grammar symbol A and category a ; $rev(rhs)$ pushes its replacement backwards. An arrow reads “input, stack top / replacement”: $a, X / X$ reads a and keeps X . The **rightmost** stack symbol is the top.

A simple successful run

For *je suis kass* (“I am tired”), the categories are **FF**, **V**, **ADJ**: French function word, verb, adjective. We group the intermediate grammar expansions below.

Stage	In simple language
Prepare	Put Ut on the stack and apply rules until FF is expected.
Read <i>je</i>	Match FF . The next rules prepare for a verb.
Read <i>suis</i>	Match V . The next rule allows an adjective.
Read <i>kass</i>	Match ADJ . There are no more input tokens.
Finish	Remove empty-rule symbols. End markers agree: accept .

Rejected example: *je suis alli* stops at token 3: *alli* is UNKNOWN, with no applicable grammar rule. It is not skipped.

The difference to remember

Lexer: “What are the pieces?” **Parser:** “Do those pieces fit our grammar?”

Accept means grammar fit, not a guarantee of meaning.

Main files: `compiler/parser/predictive.py` (stack machine), `compiler/parser/yaounde.py` (corpus grammar).